

World Models

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

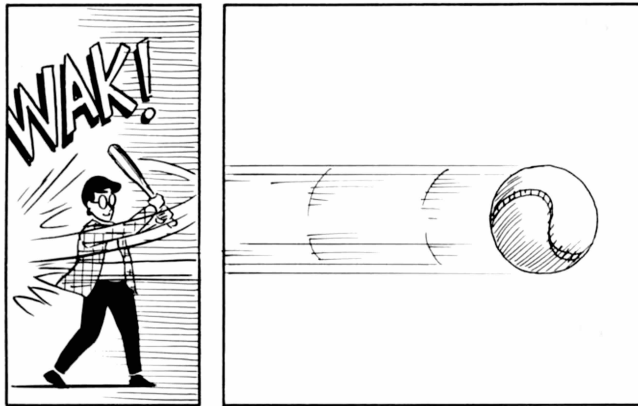


LMH
Lady Margaret Hall
University of Oxford

June 12, 2026

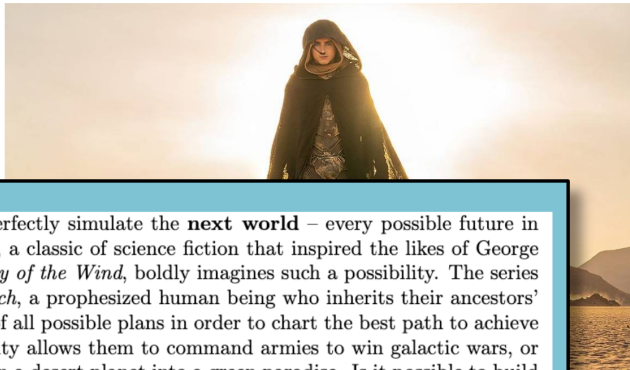
1. World models: motivation, definition, applications, and modelling paradigms
2. Ha & Schmidhuber (2018): V-M-C architecture (VAE vision, MDN-RNN memory, controller)
3. RSSM / PlaNet: stochastic and deterministic latents, ELBO, train vs. imagine
4. Dreamer $\rightarrow$ DreamerV2 $\rightarrow$ DreamerV3: imagined rollouts and scaling
5. Generative simulators vs. JEPA: architectures, V-JEPA, and V-JEPA 2
6. LeWorldModel (LeWM): end-to-end JEPA from pixels, SIGReg, and latent planning
7. Summary and references

Intuition: a baseball batter reacts in milliseconds—action is tied to a **predictive**, action-conditioned internal model, not to slow deliberate search.



Scott McCloud excerpt (time-space in comics).

Dune offers one notion of a world model...

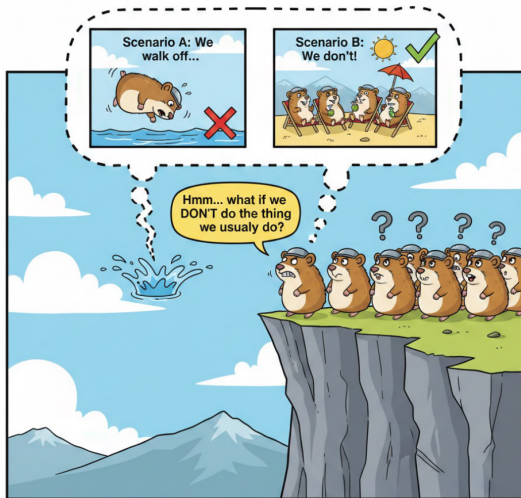


What would you do if you could perfectly simulate the **next world** – every possible future in the environment that we reside? *Dune*, a classic of science fiction that inspired the likes of George Lucas' *Star Wars* and Miyazaki's *Valley of the Wind*, boldly imagines such a possibility. The series is centered around the *Kwisatz Haderach*, a prophesized human being who inherits their ancestors' memories and simulates the outcomes of all possible plans in order to chart the best path to achieve their goals [25]. Such superhuman ability allows them to command armies to win galactic wars, or to oversee global-scale projects that turn a desert planet into a green paradise. Is it possible to build towards computer systems with similar functionalities using a similar approach?

Text from <http://arxiv.org/abs/2507.05169>

the psychology literature has long defined our notion of a world model as “hypothetical thinking”

informally, we speak about this as a “thought experiment”



- ▶ Humans develop a mental model of the world based on what they are able to perceive with their limited senses. Our decisions and actions are based on this internal model.
- ▶ Train a generative world model in an unsupervised way to learn a compact spatio-temporal representation of the environment.
- ▶ Use features from the world model to train a simple policy (agent) to solve the required task.

Definition: A *world model* takes a previous world state s and an action a , and samples (or predicts) the next world state s' through a distribution (or function):

$$s' \sim p(s' \mid s, a)$$

Question: What does a world state hold?

Answer: It depends on what you're trying to model. Consider the contrast of:

- A) All geopolitical entities, their leaders, and decisions they make
- B) Physical interactions in a 3D world

- ▶ A **world model** is a learned map that summarizes experience so we can predict (or sample) **what happens next** and sometimes **counterfactuals**: “if I had pushed left ...”
- ▶ An **agent** chooses actions a_t and receives observations o_t and rewards r_t . Planning and learning are easier if we can **imagine** plausible futures without acting in the real environment.
- ▶ Core tension: predict **everything in pixels** (high-dimensional, noisy) vs. predict in a **compact latent space** (lose detail but gain stability and generalization).

How could we use a World Model (WM)?

1. Generate (simulated) training data for robotics
2. Generate (simulated) training data for autonomous driving
3. Create interactive worlds (aka. computer games) through prompting
4. Enable automatic generation / editing / refinement of 3D animation or graphics
5. Allow a thinking model to hypothesize about physical interactions in the real world
6. Enable faster creation of environments for virtual reality or augmented reality

Some paradigms of World Models

Generate a 3D Scene, then Render/Simulate



Interactive Videos



Latent World Representations



Stanford 10-423, Lecture 26.

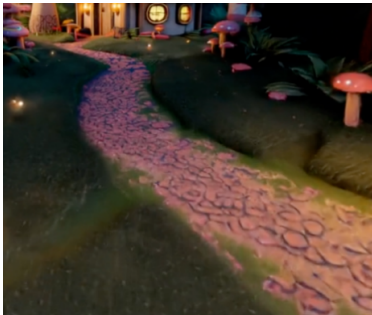


IMAGE CREDITS: WORLD LABS

Fei-Fei Li's World Labs speeds up the world model race with Marble, its first commercial product

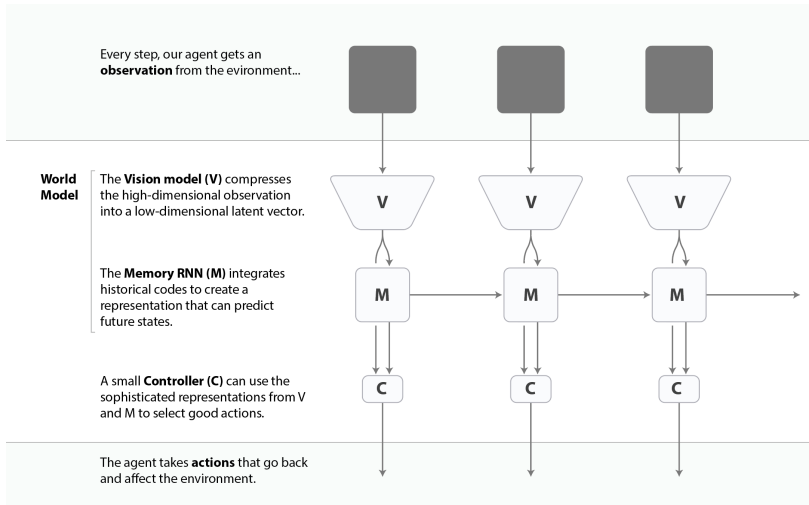
Rebecca Bellan — 5:44 AM PST · November 12, 2025

[World Labs](#), the startup founded by AI pioneer [Fei-Fei Li](#), is launching its first commercial world model product. [Marble](#) is now available via freemium and paid tiers that let users turn text prompts, photos, videos, 3D layouts, or panoramas into editable, downloadable 3D environments.

StrictlyVC an endeavor

May 27 | Athens, Greece

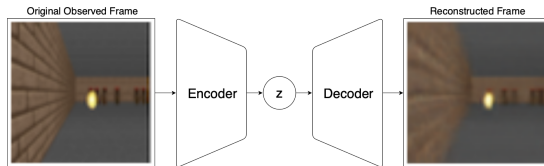
World Labs.



World model overview.

Source: Ha & Schmidhuber, 2018

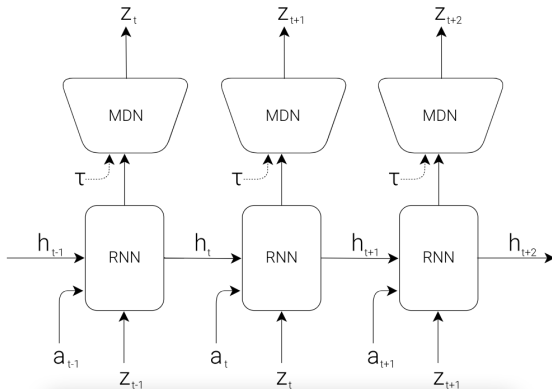
- ▶ **Goal:** learn an abstract, compressed representation of each observed input frame.
- ▶ **VAE:** first stage of a two-stage sequence model—similar in spirit to modern tokenizers, but trained with classic VAE objective.



VAE diagram.

Source: Ha & Schmidhuber, 2018

- ▶ Model $p(z_{t+1} | z_t, a_t, h_t)$ where z_t is the latent state at time t , a_t is the action at time t , and h_t is the hidden state at time t .
- ▶ adjust a temperature parameter τ to control model uncertainty



MDN-RNN architecture.

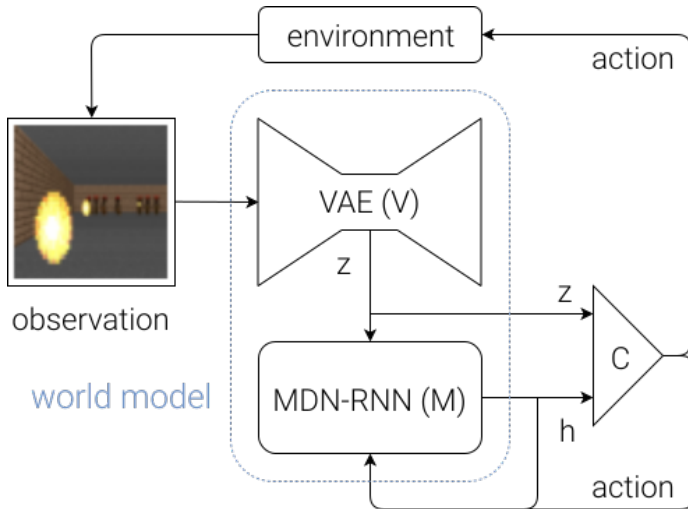
Source: Ha & Schmidhuber, 2018

- ▶ The **Controller (C)** selects actions to maximize expected cumulative reward during agent rollout.
- ▶ Designed to be as simple and small as possible; trained **separately** from the Vision (V) and Memory (M) modules.
- ▶ Most agent complexity is thus concentrated in the world model (V and M).
- ▶ C is a **single-layer linear** model that maps the latent vector z_t and RNN hidden state h_t directly to the action a_t at each time step:

$$a_t = W_c[z_t; h_t] + b_c$$

- ▶ Here, W_c and b_c are the weight matrix and bias; $[z_t; h_t]$ is the concatenated latent and memory state.
- ▶ Output vector a_t is the action taken by the agent.

World model: Putting Everything Together

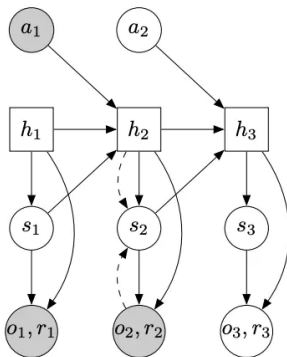


Agent-environment flow (observation $\rightarrow z_t$, RNN state h_t , action a_t , next step).

Source: Ha & Schmidhuber, 2018

- ▶ **Separation of concerns:** perception (VAE), dynamics (RNN), decision (controller)—easy to prototype but may **not** share gradients end-to-end through the full agent loop at scale.
- ▶ **Stochastic latents** matter: the world is multi-modal; Gaussian or MDN heads encode “many possible futures”.
- ▶ Next step (PlaNet, Dreamer): unify training with **RSSM**-style latent dynamics and **model-based RL** losses.

- ▶ Purely deterministic RNNs (e.g., GRUs, LSTMs) struggle to represent multiple plausible futures in complex, stochastic environments.
- ▶ **Recurrent State Space Models (RSSM)** address this by combining:
 - **Deterministic hidden state** h_t (e.g., GRU): maintains long-term memory and context.
 - **Stochastic state** z_t (e.g., Gaussian): captures uncertainty and enables diverse possible futures.
- ▶ This split allows RSSM to unify stable memory (for causality) and flexible modeling (for multi-modal, uncertain environments).
- ▶ Without the stochastic component, agents often **fail to learn** in partially observable or noisy tasks.
- ▶ Result: a flexible temporal representation for RL agents capable of handling uncertainty and representing many possible outcomes.



Recurrent State Space model (RSSM). The filled circles are observed variables. Solid lines denote the generative process and dashed lines denote the inference model. Circles represent stochastic variables, squares represent deterministic variables. This is a mixed deterministic stochastic model Source: <https://arxiv.org/pdf/1811.04551>.

- **Posterior transition (training):** Updates stochastic state using real observation

$$z_t \sim q_\phi(z_t \mid h_t, x_t)$$

- **Prior transition (imagination):** Predicts next latent state without observation,

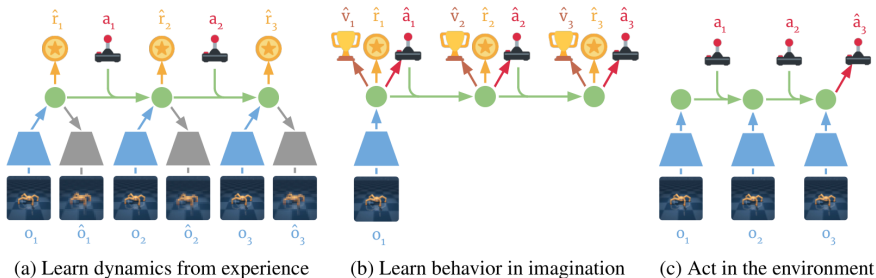
$$z_t \sim p_\theta(z_t \mid h_t)$$

- **Training objective:** Model learns by trying to reconstruct images, predict rewards, and making sure its imagined predictions match what really happened. Uses an loss (ELBO) that combines reconstruction, reward, and a KL-divergence.
- This ensures that planning in latent space is meaningful: predictions made during “imagination” are grounded in real transitions.

Model-based RL agent with RSSM core that achieves state-of-the-art performance on image-based control tasks but requires fewer samples.



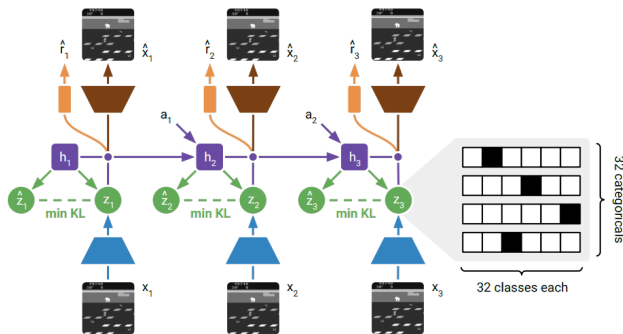
Hafner et al., *Dream to Control*. Chen, *Towards AI* (2020).



Dreamer architecture. Diagram credit: Hafner et al. (ICLR 2020), [PDF](#)

Dreamer is composed of three main parts:

- **(a) Dynamics Learning**: learns representation, transition, reconstruction, reward models.
- **(b) Behavior Learning**: based on the dynamics model, learns actor-critic architecture to maximize rewards.
- **(c) Environment Interaction**: using action model for data collection.



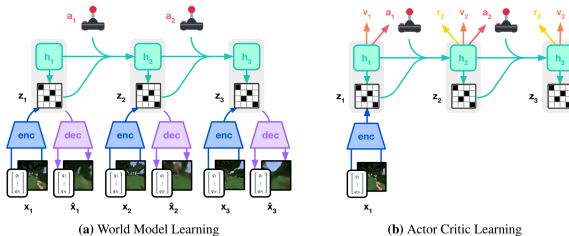
World-model overview. Diagram credit: Belotti (2023)

- ▶ Model components are exactly the same as those in Dreamer-V1.
- ▶ The only notable difference: **model size** increases—the number of dense units in the MLPs and feature maps per layer are larger in all model components.
- ▶ This change results in an increase from **13M** parameters (V1) to **22M** parameters (V2).

- ▶ Achieves state-of-the-art performance across 150+ diverse tasks with a single configuration.
- ▶ Learns a world model to imagine future trajectories. Successfully solves challenging open-world environments such as collecting diamonds in Minecraft without human data.
- ▶ Architecturally similar to DreamerV2. Robustness and scalability stem from careful normalization, balancing, and mathematical tricks that enable stability across domains.

Learning algorithm consists of three neural networks:

- **World model:** RSSM model that predicts the outcomes of potential actions, modeling future states of the environment.
- **Critic:** Judges the value of each predicted outcome, estimating how beneficial a state is for achieving the agent's goals.
- **Actor:** Selects actions intended to achieve the most valuable outcomes by maximizing the evaluated value.



Hafner et al., [arXiv:2301.04104](https://arxiv.org/abs/2301.04104).

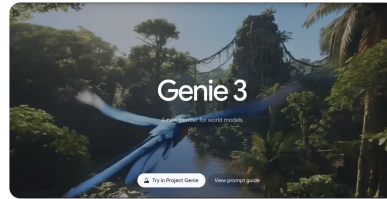
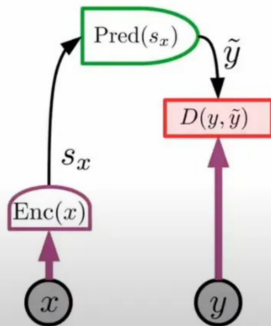


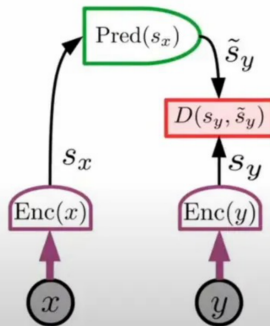
Image credit: CS25.

Architectures: Generative v/s Joint Embedding

- ▶ **Generative:** predicts y (with all the details, including irrelevant ones)
- ▶ **Joint Embedding:** predicts an **abstract representation** of y

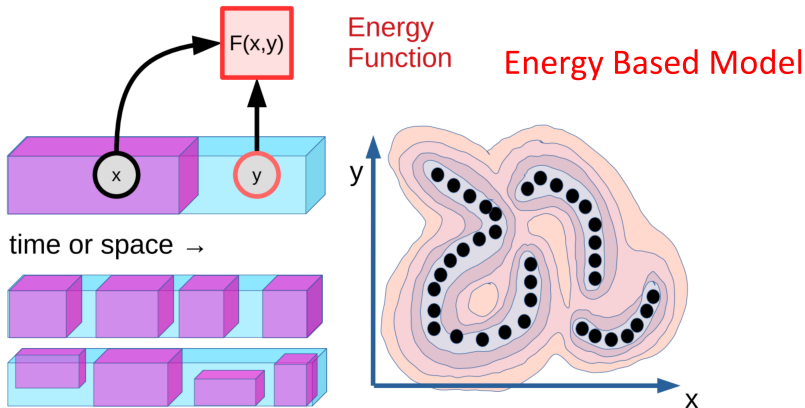


a) Generative Architecture
Examples: VAE, MAE...

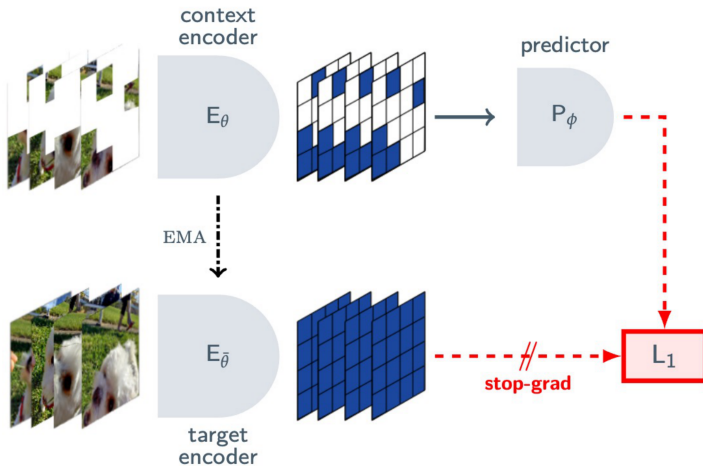


b) Joint Embedding Architecture

Image credit: CS25.

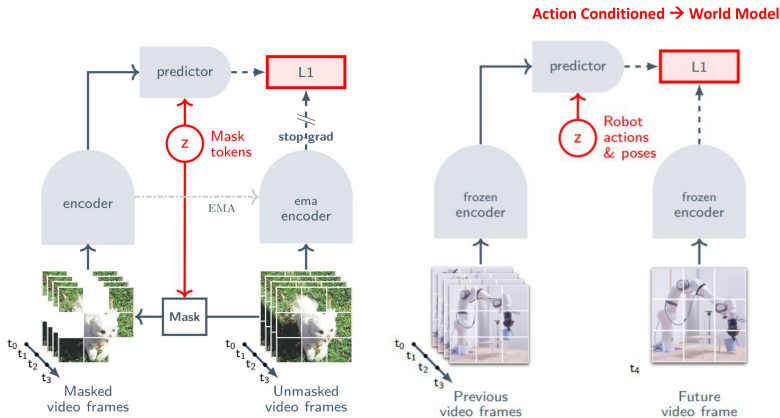


Paper: [OpenReview](#), Y. LeCun (2022).



Paper: Bardes et al., [arXiv:2404.08471](https://arxiv.org/abs/2404.08471) (V-JEPA).

V-JEPA 2: action-conditioned latent prediction



Paper: Assran et al., [arXiv:2506.09985](https://arxiv.org/abs/2506.09985) (V-JEPA 2).

Generative (reconstruction) path

- ▶ Learn latents that **decode** back to pixels (VAE/RSSM).
- ▶ Strength: full **observation model** for planning, denoising, and imagined rollouts with explicit images.
- ▶ Cost: optimization can emphasize pixels over semantics.

Joint-embedding predictive (JEPA) path

- ▶ Predict **embeddings** of future or masked regions in **representation space**.
- ▶ Strength: can learn useful **video/robotics** features without needing to generate every pixel.
- ▶ Cost: requires clear action input (e.g. V-JEPA 2-AC, LeWM).

LeWorldModel (LeWM): Motivation

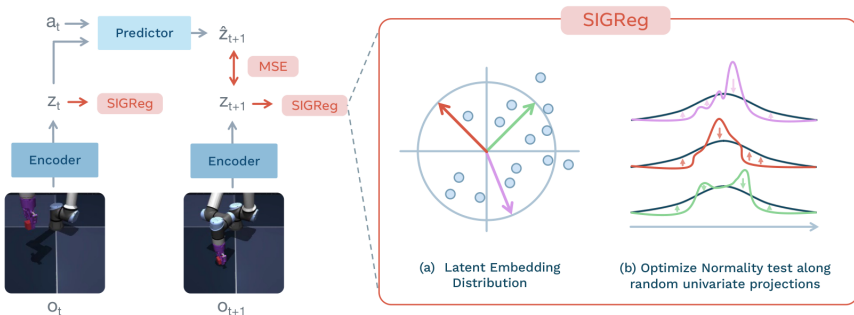


Image credit: Maes et al., [LeWorldModel \(arXiv:2603.19312\)](#).

- **LeWorldModel (LeWM):** The LeWorldModel is the first JEPA-style world model trained end-to-end from raw pixels using only two simple loss terms: a prediction loss and a regularizer that encourages Gaussian-distributed latents. This streamlined approach avoids the complex loss functions and additional tricks used by earlier models, yielding a provably stable and anti-collapse training procedure.

LeWorldModel (LeWM): Architecture

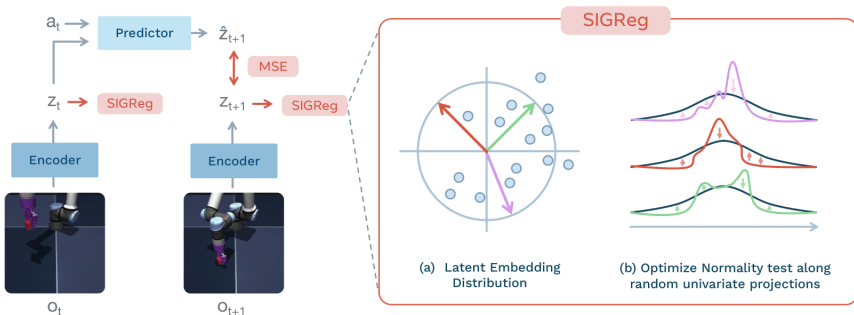
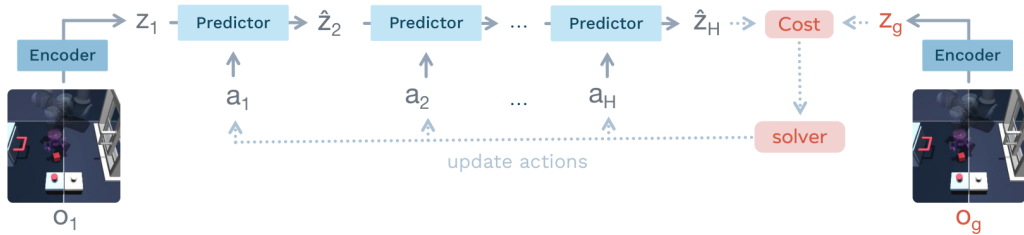


Image credit: Maes et al., [LeWorldModel \(arXiv:2603.19312\)](#).

- **Model Architecture.** LeWM consists of an encoder (mapping frames to latent codes) and a predictor (forecasting the next latent given the current latent and action).
- **Training Objective.** The model is trained end-to-end with a prediction loss plus a regularizer that encourages Gaussian-distributed latents:

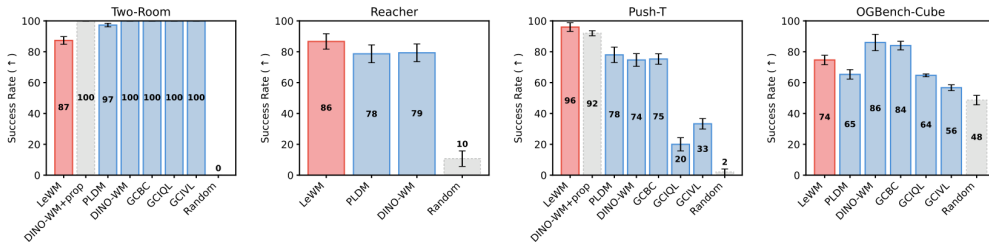
$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \text{SIGReg}(Z)$$

LeWorldModel (LeWM): Latent Planning



LeWorldModel Latent Planning. Given an initial observation o_1 and a goal o_g , the world model learned performs planning in the LeWM latent space. The initial state embedding z_1 and the goal embedding z_g are obtained from the encoder. The predictor then rolls out future latent states up to a horizon H . A latent cost between the final predicted state and the goal embedding guides a solver to optimize the action sequence. This prediction–optimization loop is repeated until convergence to a good plan candidate.

LeWorldModel (LeWM): Results



Planning performance across environments. Results are shown for *Two-Room* (left), *Reacher* (center 1), *PushT* (center-2) and *OGBench-Cube* (right). LeWM consistently outperforms PLDM and DINO-WM on PushT and Reacher. On OGBench-Cube, DINO-WM slightly outperforms LeWM, possibly due to the higher visual complexity and the 3D nature of the environment, which makes encoder training more challenging. In the simpler Two-Room environment, PLDM and DINO-WM outperform LeWM, which may be explained by the SIGReg regularization encouraging a Gaussian distribution in a high-dimensional latent space, while the intrinsic dimensionality of the environment is much lower.

- ▶ World models compress experience so an agent can roll imagined futures and choose actions; the main design split is predict pixels (generative simulators) vs. predict latents (JEPA-style), with hybrid systems in between.
- ▶ Dreamer → DreamerV2 → DreamerV3: a single RSSM-style world model (representation, dynamics, often decode/reward) plus actor-critic training on imagined trajectories; V2 scales width; V3 keeps the recipe and stresses stability across many domains with one configuration.
- ▶ JEPA: predict in embedding space, not full observations; V-JEPA scales masked video prediction; V-JEPA 2 adds action-conditioned latent prediction as a bridge to planning and robotics.
- ▶ LeWorldModel: End-to-end from pixels with just next-embedding loss + SIGReg. Fast latent MPC planning with a small model.

1. Pedagogical synthesis (informed this revision): *Architectural Evolution of Latent World Models: A Synthesis of Generative and Joint-Embedding Predictive Frameworks* (2026). Raster excerpts: `assets/New-Lecture_World_Models/latent_wm_synthesis_report_p{1--4}.png`.
2. Schmidhuber, formative work on **curiosity** and agents that build **internal predictive models** (1990s–2000s).
3. Ha & Schmidhuber, *Recurrent World Models Facilitate Policy Evolution* (NeurIPS 2018), [arXiv:1803.10122](#); interactive write-up [worldmodels.github.io](#).
4. Gormley, M., CMU 10-423 / 10-623, **Lecture 26: World Models** (slides as `cmu10423_lecture26_world_models_slide03.pdf`). [PDF](#); F25 mirror [PDF \(10423-F25\)](#).
5. Stanford CS25 deck excerpt (**JEPA / V-JEPA** slides raster): `jepa_cs25_sl04–sl08_*.png` from [JEPA_cs25.pdf \(Google Drive\)](#); course [CS25](#).
6. Stanford CS25 deck excerpt (**Joint Embedding Predictive World Models** slides raster): `jepa_cs25_sl04–sl08_*.png` from [JEPA_cs25.pdf \(Google Drive\)](#); course [CS25](#).
7. Cambridge teaching archive (Ha 2018 PDF). [cl.cam.ac.uk PDF](#).
8. Hafner et al., “Learning Latent Dynamics for Planning from Pixels” (PlaNet), [arXiv:1811.04551](#). *RSSM plate*: `planet_rssm_page_3.png`.
9. Hafner et al., *Dream to Control: Learning Behaviors by Latent Imagination* (ICLR 2020), [arXiv:1912.01603 \(Dreamer\)](#). *Slides*: Figs. 1,3,4,8 cropped from [PDF](#).
10. Hafner et al., “Mastering Atari with Discrete World Models” (**DreamerV2**), [arXiv:2010.02193](#). Pedagogical diagrams: Belotti, [Dreamer V2](#) (EclecticSheep blog, 2023); `eclecticsheep_dreamerv2_worldmodel_arch.png`, `eclecticsheep_dreamerv2_straight_through_gradients.png`.